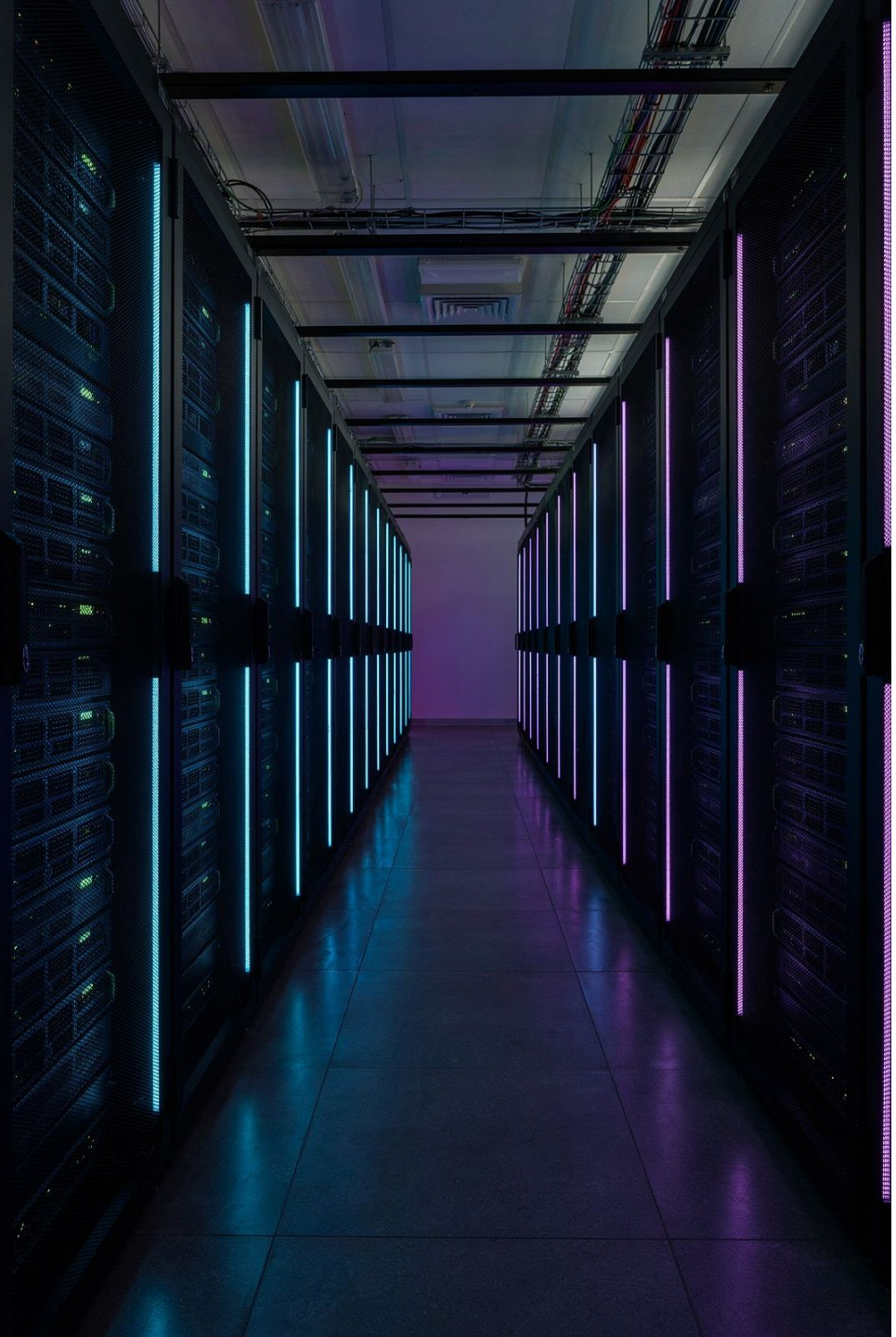


GPU Computing on AWS

From raw compute power to production-ready LLM inference — a practical guide to running GPU workloads on Amazon Web Services using EC2, EKS, and modern open-source tooling.

Why GPUs? The Real-World Need

The Problem

Training a large language model or running real-time image generation on a regular CPU would take **weeks or months**. A task that takes 10 minutes on a GPU could take 10 days on a CPU.

Think of a CPU as a single expert doing tasks one by one. A GPU is a stadium full of workers doing everything in parallel.

Real-World Use Cases

- 🤖 Chatbots powered by large language models (e.g., GPT-style)
- 🎨 Text-to-image generation (e.g., Stable Diffusion)
- 🧬 Drug discovery and protein folding simulations
- 📊 Real-time fraud detection at scale
- 🎬 Video transcoding and content analysis

AWS EC2 GPU Instance Families

AWS offers purpose-built GPU instance families for different workload sizes and budgets. Think of them as small, medium, and large GPU "rental plans."

G5 — The Everyday Workhorse

Powered by **NVIDIA A10G GPUs**. Best for model inference, game streaming, and graphics rendering. Great starting point — cost-effective and widely available.

💡 Example: Hosting a Stable Diffusion API for image generation.

P4d — The Training Powerhouse

Powered by **NVIDIA A100 GPUs** with 400 Gbps networking. Built for distributed deep learning training across multiple nodes.

💡 Example: Fine-tuning a 70B parameter language model on proprietary data.

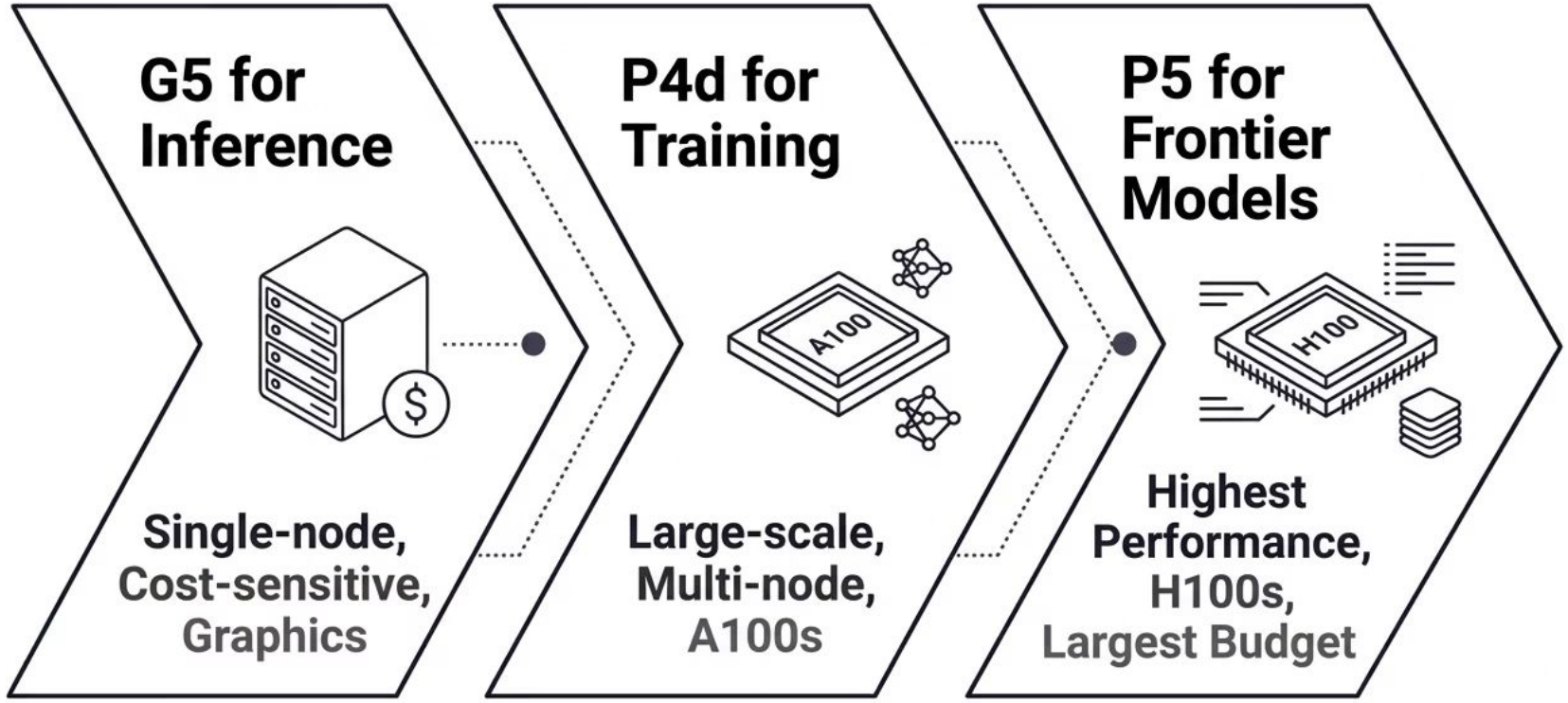
P5 — The Frontier Machine

Powered by **NVIDIA H100 GPUs** — the latest and most powerful. Ideal for cutting-edge foundation model training at hyperscale.

💡 Example: Pre-training a GPT-4-class model from scratch.

Choosing the Right Instance

Picking the wrong instance is like renting a stadium when you need a conference room — or vice versa. Here's a simple decision guide.



Amazon EKS: Orchestrating GPU Workloads

Running a single GPU job manually is easy. Running hundreds of GPU jobs reliably — scheduling, scaling, and recovering from failures — requires an orchestrator. That's where **Amazon EKS (Elastic Kubernetes Service)** comes in.

What EKS Does

EKS is a managed Kubernetes service. Think of Kubernetes as an air traffic controller for your compute jobs — it decides which job runs on which machine, when, and how many resources it gets.

- Schedules GPU pods to GPU-capable nodes automatically
- Restarts failed workloads without manual intervention
- Manages multiple teams sharing the same cluster

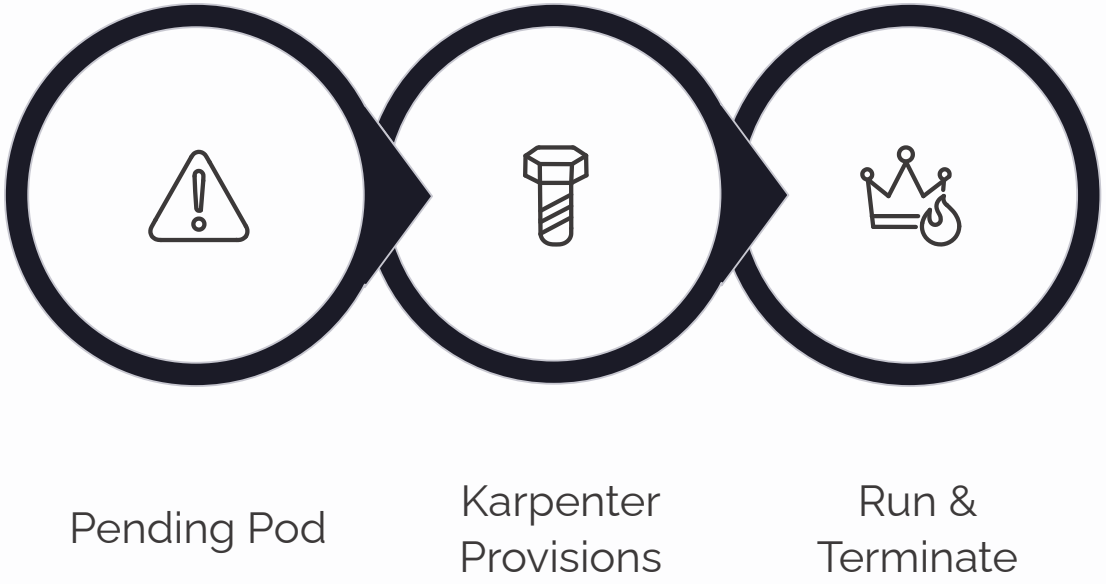
GPU Node Groups

A **GPU Node Group** is a pool of EC2 GPU instances (e.g., G5 or P4d) attached to your EKS cluster. You define the instance type, minimum, and maximum node count.

- Dedicated pools for training vs. inference workloads
- Spot instance support for up to 90% cost savings
- Node taints ensure only GPU jobs land on GPU nodes

Karpenter: Smart, Instant Autoscaling

Traditional autoscaling is slow — it can take 10–15 minutes to add a new node. **Karpenter** is a modern, open-source autoscaler built for Kubernetes that provisions nodes in under 60 seconds.



Why Karpenter Beats Traditional Autoscaling

- Provisions the **exact right instance** for the workload — no over-provisioning
- Supports GPU instance diversity to maximize Spot availability
- Automatically consolidates underutilized nodes to cut costs

Real-World Impact

Imagine a batch inference job that suddenly needs 20 A10G GPUs. Karpenter spins up G5 instances in under a minute, runs the job, and terminates the nodes when done — you only pay for what you use.

NVIDIA GPU Operator on EKS

Getting Kubernetes to actually *talk to* GPUs requires special software drivers and plugins. Without them, your cluster sees a GPU node as just another CPU machine. The **NVIDIA GPU Operator** automates all of this setup.



What It Installs Automatically

GPU drivers, the NVIDIA Container Runtime, the Kubernetes device plugin, and DCGM monitoring — all deployed via Helm in minutes, not days of manual configuration.



GPU Monitoring Built In

DCGM (Data Center GPU Manager) exports metrics like GPU utilization, memory usage, and temperature directly to Prometheus and Grafana — so you always know how your GPUs are performing.



Automatic Driver Updates

When a new GPU driver is released, the Operator handles the rollout across all nodes. No SSH, no manual patching — just a version bump in your Helm values file.

KServe: Serving LLMs at Scale on EKS

Once your model is trained, you need to serve it reliably to thousands of users. **KServe** is a Kubernetes-native model serving framework designed specifically for this — think of it as the "waiter" that takes user requests and delivers model responses efficiently.

How KServe Works for LLM Inference

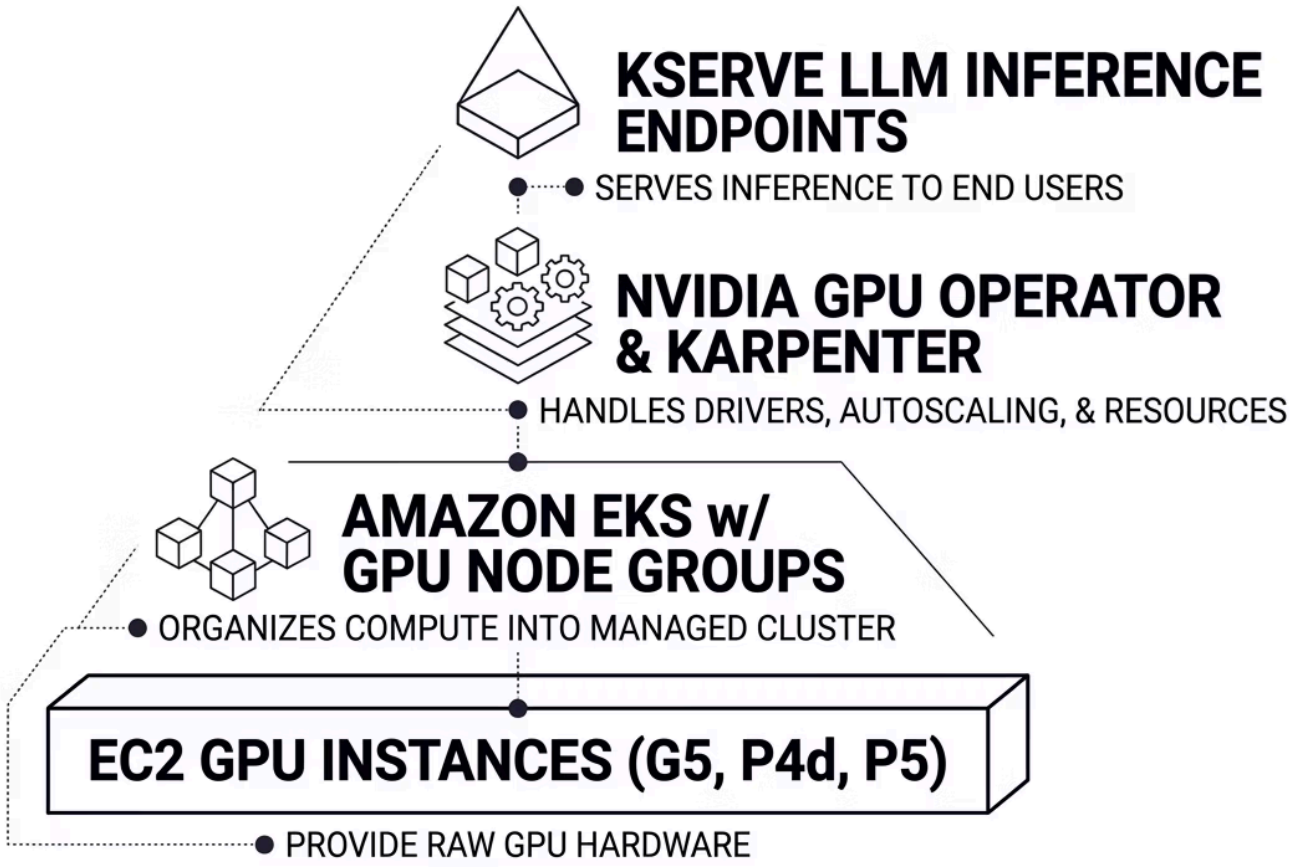
- Deploys models as scalable Kubernetes services with a single YAML file
- Supports popular runtimes: **vLLM, Triton, TorchServe, ONNX**
- Automatic batching groups multiple user requests into one GPU call — dramatically improving throughput
- Scales inference pods to zero when idle, saving GPU costs overnight
- Built-in canary deployments — safely roll out new model versions to 10% of traffic first

Real-World Example

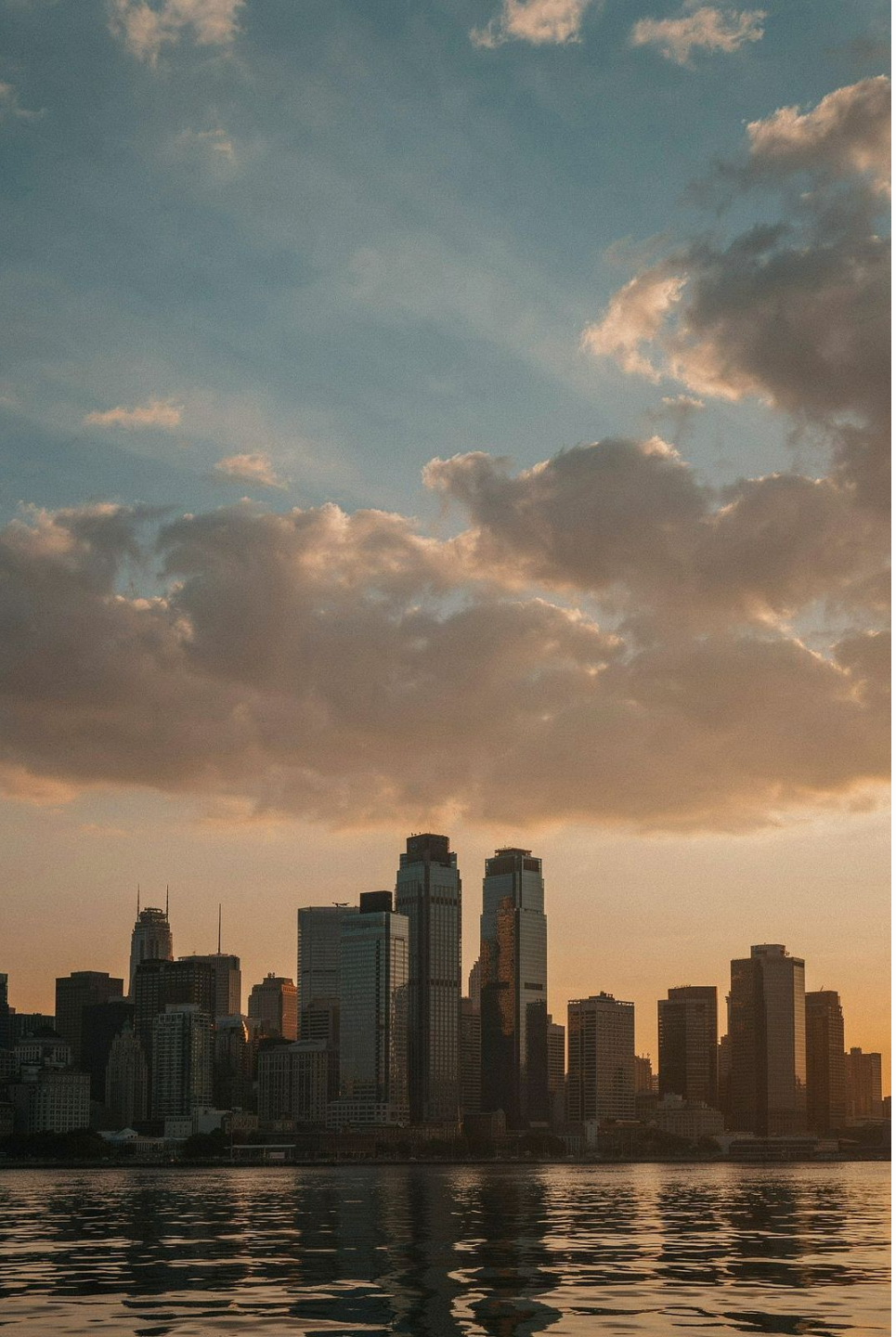
A company deploys **Llama 3 70B** on KServe backed by G5.48xlarge instances. During business hours, KServe auto-scales to 8 replicas. At 2 AM, it scales down to 1. Users get fast responses; the company saves 60% on overnight GPU bills.

The Full Stack: How It All Fits Together

Each layer plays a distinct role — from raw hardware to intelligent request routing. Here's how the complete solution looks end-to-end.



Each layer builds on the one below it. You can adopt them incrementally — start with a single G5 instance, add EKS, then Karpenter, then KServe as your needs grow.



Key Takeaways & Where to Start

1

Pick the Right GPU Instance

Start with **G5** for inference and experimentation. Graduate to **P4d or P5** when you're training large models.

2

Use EKS + Karpenter for Scale

Don't manage GPU servers manually. Let EKS schedule workloads and Karpenter provision exactly the right nodes — automatically and cheaply.

3

Install the GPU Operator First

It's the foundation. Without it, Kubernetes can't use your GPUs. Install it via Helm before anything else.

4

Serve Models with KServe

For production LLM inference, KServe gives you auto-scaling, batching, and multi-model serving out of the box — without building custom API servers.